Computational Problems

February 17, 2026

1 Introduction

In this lecture we talk about computational problems, defining them in terms of input and output, internal representation of data, and algorithms. We classify computational problems using several criteria and discuss their relevance in real-world situations.

Every day we are confronted with an abundant number of problems to solve. For some of them, we may have established routines (algorithms), or default solutions, while others are completely new and may require more effort or creativity to solve. We can think of ourselves as "machines" that solve various problems.

One frequent problem is getting to classes in the morning. In order to solve this problem, there are several subproblems that we need to tackle: waking up in time, getting ready for classes, and arriving at the uni. Depending on how lightly one sleeps, the waking up in time may be solved with a single alarm ringing at the same time every morning (or even better waking up without an alarm thanks to the inner clock), or it may imply multiple alarms that are snoozed several times or even not heard at all. In a similar fashion, all other subproblems that we need to tackle can have very different solutions (whereby solution we understand the set of steps that bring us to completing the task at hand), depending on inner preferences, struggles, unforeseen changes etc.. Even if the problem seems to be the same, the input changes (it's one thing to wake up at 7AM when you go to bed at 10PM, and a different thing to wake up at 7AM when you go to bed at 1AM, or 3AM, or 5AM). So, correctly identifying the problem, its inputs and the expected output is crucial to solving the problem. We will discuss in Section 2 about defining a problem as an Input-Output pair.

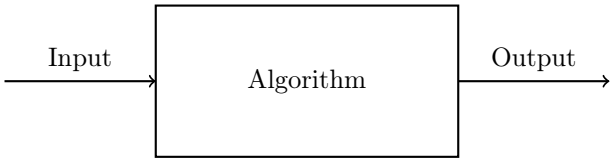
2 Defining computational problems

A problem is a computational problem if it can be solved using some mode of computation (for example, a Turing Machine). We can think of a problem as a relation between inputs and outputs, or as a function mapping inputs (problem instances) to correct outputs.

"A computational problem is specified by a set of instances together with a function that maps each instance to a correct output." [2]

"A computational problem specifies the desired relationship between inputs and outputs, and an algorithm provides a step-by-step method for achieving this relationship." [3]

Formally, we will consider a problem to be a function $f : I \to O$ that maps each valid input to a valid output, where I is an input set, and O is an output set.



There are several reports of Einstein being asked: **"If you have one hour to save the world, how would you spend that hour?"**, with different reported answers. The most common is "I would spend 55 minutes defining the problem and then five minutes solving it." Another report claims he responded he would spend 30 minutes analyzing the problem, 20 minutes planning the solution, and ten minutes executing the solution. A third report claims the answer to be "If I were given an hour in which to do a problem upon which my life depended, I would spend 40 minutes studying it, 15 minutes reviewing it and 5 minutes solving it."

The point we are trying to make here is that **understanding the problem is crucial to correctly solving it**. Spending time reviewing (improving) an initial idea for a solution may lead to finding a more cost-effective solution, and is thus worthwhile.

A problem P to be solved by an algorithm is represented by a pair (INPUT, OUTPUT).

We will denote by p an **instance** of problem P (a valid input for problem P), and by $P(p)$ the output corresponding to instance p.

INPUT: describes the input data

OUTPUT: describes the output data (depending on input data)

When defining a computational problem as an (INPUT, OUTPUT) pair, we can describe the input and output data more coarsely. The exact representation of input and output data becomes relevant when implementing an algorithm to solve a given problem. In specifying the input and output of a problem, we can use natural language, but it is better to define Boolean predicates to specify properties that the data satisfy.

Example: Greatest Common Divisor (GCD): given two natural numbers a and b, what is their greatest common divisor (factor)?

GCD *(in natural language)*
INPUT: Two numbers $a, b \in \mathbb{N}$;
OUTPUT: A natural number d that is the greatest common divisor of a and b.

Let us remember what a common divisor c of two numbers a and b means: c is a number that is a divisor of both a and b.

We can define a predicate $\mathrm{D}(x, y)$: $y \text{ MOD } x = 0$ to specify that x divides y, and use it twice to represent the property of a common divisor of two numbers: $\mathrm{CD}(c, a, b)$: $\mathrm{D}(c, a) \wedge \mathrm{D}(c, b)$.

We are searching for the **greatest** common divisor, so we need to specify this property too. A number n is the greatest one satisfying a property $\mathrm{P}(x)$ if any other number with that property is less than or at most equal with it. We can define a predicate as follows: $\mathrm{GP}(n)$: $\mathrm{P}(n) \wedge \forall m.\mathrm{P}(m) \rightarrow m \leq n$. Let us go back to the definition of the GCD problem, and specify the output using predicates.

GCD *(formalized using logical predicates)*
INPUT: Two numbers $a, b \in \mathbb{N}$;
OUTPUT: A number $d \in \mathbb{N}$ such that $\text{CD}(d, a, b) \wedge (\forall c.\text{Nat}(c) \wedge \text{CD}(c, a, b) \rightarrow d \geq c)$, with
predicates $\text{Nat}(x)$: $x \in \mathbb{N}$, and $\text{CD}(x, y, z)$: $y \text{ MOD } x = 0 \wedge z \text{ MOD } x = 0$.

Another alternative is using mathematical notations to specify the properties that the input and output must satisfy.

GCD *(formalized using mathematical notations)*
INPUT: Two numbers $a, b \in \mathbb{N}$;
OUTPUT: A number $d \in \mathbb{N}$ such that $a \vdots d$ and $b \vdots d$ and $\forall c \in \mathbb{N}$ s.t. $a \vdots c$ and $b \vdots c$, $c \leq d$.

When defining a problem as an INPUT-OUTPUT pair, aim to be formal and use logical predicates or mathematical notations!

3 Classifying computational problems

We can discuss computational problems from multiple perspectives:

- What is the output required?
- What type of input data does the problem use?
- What resources are required to solve it? Can it be solved at all?

3.1 Classification based on output type

Depending on the nature of the expected output, we can classify computational problems as: decision problems, optimization problems, search problems, counting problems, etc.. More broadly, we can think of problems as returning a binary output to answer a yes/no question (decision problems), or computing/returning something else.

3.1.1 Decision problems

A decision problem requires a simple Boolean answer (Yes/No, True/False, 0/1), and is usually formulated as a question. Decision problems appear in numerous real-world applications across different domains. They are often the basis for more complex problems, including optimization problems and AI-based systems.

Formally, a **decision problem** is a function $f : I \rightarrow \{\text{Yes, No}\}$ where:

- I is the set of all possible problem instances;
- $f(x) = \text{Yes}$ if the instance x satisfies the given property;
- $f(x) = \text{No}$ otherwise.

Example: Primality Testing: given a number, is it prime or not?

PRIMES *(in natural language)*
INPUT: A number $k \in \mathbb{N}$;
OUTPUT: Yes, if k is prime; No, if k is compound or 1.

There are multiple ways of expressing the property of being a prime number. The easiest way is to specify that a number k is prime if it has no proper divisor (it is not divided by any number $1 < i \leq k$; and you may recall from mathematics that if a number has no divisors up to $\sqrt{k}$, then it is prime).

PRIMES *(formalized)*
INPUT: A number $k \in \mathbb{N}$;
OUTPUT: $\begin{cases} \text{Yes, if } k \text{ MOD } i \neq 0, \quad \forall i \in \mathbb{N}, \quad 1 < i \leq \sqrt{k}, \\ \text{No, if } \exists a, b \in \mathbb{N},\ a, b \neq 1 \text{ s.t. } k = a * b. \end{cases}$

Decision problems can also be specified as INSTANCE-QUESTION pairs, where INSTANCE gives the input of the problem, and QUESTION is the question that the problem answers.

PRIMES
INSTANCE: A number $k \in \mathbb{N}$;
QUESTION: Is k prime?

Example: Hamiltonian circuit: given a directed graph, does it contain a Hamiltonian circuit (a circuit that visits every vertex exactly once)?

HAMILTONIAN-CIRCUIT
INSTANCE: a directed graph $G = (V, E)$;
QUESTION: is there a Hamiltonian circuit in G (a permutation $v_1, v_2, \ldots v_n$ of the vertices in V such that $v_i, v_{i+1} \in E$ $(1 \leq i < n)$ and $v_n, v_1 \in E$))?

Example: Boolean Satisfiability (SAT): Given a Boolean formula, is it satisfiable?

SAT
INSTANCE: a Boolean formula φ in NCF;
QUESTION: is φ satisfiable (is there an assignment τ that makes the formula True)?

Real-life applications of decision problems: Password Verification, Spam Detection, Face Recognition, Disease Diagnosis, Collision Detection in Self-Driving Cars, Warehouse Robot Navigation, Loan Approval, Genetic Mutation Detection

3.1.2 Optimization Problems

An optimization problem involves finding **the best solution** according to some objective function. We can have **minimization** problems, where the objective function is minimized, or **maximization** problems, whereby one aims to obtain the maximum possible value of the objective function. The objective function can be simple, like the number of elements (e.g., edges on a path), or more complex, like a sum of elements that satisfy additional constraints.

> Generally, a **minimization** problem is defined as:
> INPUT: d;
> OUTPUT: $s \in S(d)$ s.t. $cost(s)$ is minimum (or some special marker, e.g. -1, if no solution is found).

d denotes the input data, $S(d)$ is the solution space for data d, and $cost(s)$ returns the cost of a solution (the objective function to be minimized).

> Generally, a **maximization** problem is defined as:
> INPUT: d;
> OUTPUT: $s \in S(d)$ s.t. $cost(s)$ is maximum (or some special marker, e.g. -1, if no solution is found).

Example: Shortest Path Problem: Given a graph and two nodes in that graph, find the shortest path between the two nodes (e.g., Dijkstra's algorithm, Floyd-Warshall algorithm).

> SHORTEST-PATH (minimization)
> INPUT: A graph $G = (V, E)$, two vertices $s, t \in V$;
> OUTPUT: The shortest path between s and t in G.

The solution space contains all paths between s and t.

> LONGEST-PATH (maximization)
> INPUT: A graph $G = (V, E)$, two vertices $s, t \in V$;
> OUTPUT: The longest simple path between s and t in G (no vertex should be visited more than once).

The cost of a path can be given by either the number of edges, or (in weighted graphs) by the sum of the weights of its edges.

Real-life applications of optimization problems: Vehicle Routing Problem (minimize the total distance traveled while ensuring all deliveries are made), Portfolio Optimization (given a set of stocks, determine the best way to invest money to maximize returns while minimizing risk), Inventory Management (stock shelves of a supermarket efficiently while minimizing waste), Hyperparameter Tuning in AI Models (maximize model accuracy while minimizing training time), Wireless Signal Optimization (minimize signal interference while maximizing network coverage), Path Planning for Autonomous Vehicles / Warehouse Robot Navigation (minimize the total travel time while avoiding obstacles/congestion).

3.1.3 Search Problems

A search problem requires finding at least one solution, without necessarily optimizing.

Example: Element Search: Find a given element in a sorted array.

> ELEMENT-SEARCH
> INPUT: A sorted array of data d, and a searched item k;
> OUTPUT: i, s.t. $d[i] = k$, or -1 if k is not in d.

Real-life applications of search problems: Pathfinding for Autonomous Vehicles (Self-driving cars, delivery drones), Web Search Engines, Auto-Complete & Search Suggestions, Malware Detection, DNA Sequence Search (detect genetic diseases or mutations), Product Search & Recommendations (also friend, content reccommendations), Facial Recognition Systems, Object Detection for Self-Driving Cars, Vehicle Routing, Warehouse Optimization, Plagiarism Detection, Document Retrieval Systems (Google Scholar), Fraud Detection.

3.1.4 Counting Problems

Counting problems are computational problems where the objective is to determine the number of possible solutions that meet specific criteria. Unlike decision problems (which give a Yes/No answer), counting problems focus on determining the **number of valid solutions**.

Example: Sharp Satisfiability Problem (#SAT): count the number of assignments that satisfy a given Boolean formula.

> #SAT
> INPUT: A Boolean formula φ;
> OUTPUT: $n = |T|$, where $T = \{\tau : vars(\varphi) \to \{0, 1\} \mid \hat{\tau}(\varphi) = 1\}$.

Real-life applications of counting problems: Counting Attack Vectors in Network Security (count the possible attack paths an intruder could take in a network), Counting Possible Game States, DNA Sequence Alignment (count the number of ways two DNA sequences can be aligned, to identify possible mutations and evolutionary relationships), Drug Combination Testing (count all possible combinations of drugs that can be tested together, to optimize drug testing for effective treatments), Counting Delivery Route Combinations, Inventory Management (count the total number of ways to arrange stock in a warehouse, to minimize retrieval time and optimize space usage, Graph Coloring (count the number of valid colorings of a graph with k colors to minimize resource conflicts, e.g., register allocation in compilers), Counting Paths in a Graph (analyze network reliability and data flow - internet routing), Counting Social Network Connections (to detect communities and influential users), Counting Citation Networks (to analyze research impact and trends), Plagiarism Detection (count matching sequences across different documents, to detect copied content effectively).

3.2 Classification based on the structure of input data

Depending on the nature of the input, the kind of questions we are trying to solve also differ.

3.2.1 Sorting Problems

The input of sorting problems is some form of unordered data. The problem asks to arrange the data in some specific order (increasingly, decreasingly, alphabetically, etc.). Sorting problems are vital for efficiency, organization, and optimization across nearly every industry. Efficient sorting algorithms (e.g., Merge Sort, Quick Sort, Heap Sort) are crucial for handling large datasets. Sorting is often a pre-processing step for more complex problems like search, optimization, or classification.

Real-life applications of sorting problems: Web Search Result Ranking, Auto-Complete Suggestions (sort possible word completions based on frequency and relevance), Product Sorting by Price or Popularity, Inventory Management (sort products based on expiration date), Content Feed Sorting, (sort posts, videos, or stories by relevance or time), Friend Suggestions (sort potential friends by the number of mutual connections), Delivery Route Sorting (by geographic proximity, delivery priority), Movie/Show Recommendation Sorting, Music Playlist Sorting, Exam Score Sorting

3.2.2 Graph Problems

The input for graph problems is a graph structure. Graph problems involve analyzing relationships through vertices (nodes) and edges (connections).

Example: Graph Connectivity: Given an undirected graph, are all nodes reachable from any node in the graph?

GRAPH-CONNECTIVITY
INSTANCE: an undirected graph $G = (V, E)$;
QUESTION: are all nodes reachable from any node in the graph (for a node $u \in V$, is $\mathrm{R}(u, v)$ True $\forall v \in V, v \neq u$, where $\mathrm{R}(u, v) : \exists v_0, v_1, \ldots, v_k \in V s.t. (v_i, v_j) \in E$, $\forall 0 \leq i, j \leq k, i \neq j$, and $v_0 = u$ and $v_k = v$)?

Example: Traveling Salesman Problem: Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the starting city?

TSP
INPUT: a complete graph $G = (V, E)$, a cost function $c : E \to \mathbb{N}$ (the distances between each pair of cities);
OUTPUT: $n \in \mathbb{N}$, s.t. $n = min \left(\sum_{i=1}^{n-1} c(v_i, v_{i+1}) + c(v_n, v_1) \right)$, where $v_1, \ldots, v_n$ forms a Hamiltonian circuit in G.

Real-life applications of graph problems: Shortest Path Problem (internet routing, navigation route optimization), Graph Connectivity (does a network remain connected after removing certain nodes? - ensure fault tolerance in communication networks), Community Detection in Social Networks, Influencer Detection, Graph Matching (friend suggestions), Vehicle Routing Problem (VRP), Traffic Flow Optimization (Waze real-time traffic updates), Minimum Spanning Tree (MST: connect all nodes in a graph with the least total edge weight, e.g., to desig efficient electrical grids), Graph Traversal (BFS/DFS: AI pathfinding (game development), web crawling), Cycle Detection (detect infinite loops or dependency issues), Protein Interaction Networks (for

drug discovery), Gene Regulation Networks (genetic engineering, personalized medicine), Disease Spread Modeling (to optimize vaccination strategies), Product Recommendation Graphs, Warehouse Optimization, Plagiarism Detection (graph isomorphism).

3.2.3 Geometric Problems

A geometric problem involves solving questions related to the geometry of points, lines, shapes and objects in two or more dimensions. These problems deal with spatial relationships, distances, angles, intersections, and properties of geometric figures.

> Generally, a **geometric problem** is defined as:
> INPUT: A set of geometric objects (points, lines, polygons, circles, etc.).
> OUTPUT: A numerical or structural solution (e.g., a distance, an area, or a geometric arrangement) that satisfies certain conditions.

Example: Convex Hull Problem: find the smallest convex polygon (or polyhedron in 3D) that encloses a set of points in a given space.

Real-life applications of geometric problems: Coputer Graphics (rendering realistic 3D objects on a screen), Path Planning (collision-avoidance), Geographic Information Systems (analyzing spatial data for map-based systems), Computer-Aided Design (CAD: designing and manipulating geometric models of physical objects), Wireless Networking & Coverage Optimization, Medical Imaging & Diagnostics (analyze the shape and size of tumors in medical scans), Space Exploration (calculate trajectories for satellites or space missions), Industrial Automation (optimize material usage in cutting or packing problems), Virtual Reality & Augmented Reality (render 3D objects based on a user's perspective), Real-Time Traffic Systems (prevent collisions at intersections).

3.2.4 String Processing Problems

A string processing problem involves manipulating, analyzing, or transforming sequences of characters (strings) according to specific rules. These problems are fundamental in computer science, linguistics, bioinformatics, and data processing.

The kinds of operations to be performed include: matching (checking if a pattern exists in a string), transforming (e.g., reversing, replacing some part of a string), comparing (measuring similarity or differences between strings).

Example: Pattern Matching: Find the first occurrence (/all occurrences) of a pattern P in a string S.

> PATTERN-MATCHING
> INPUT: Two strings P (pattern) and T (text), such that $\mathrm{P}.length() \leq \mathrm{T}.length()$;
> OUTPUT: $i \in \mathbb{N} \cup \{-1\}$, where $i = -1$ if pattern P does not appear in T, otherwise the index of the first occurrence of P in T.

You will study several well-known pattern-matching algorithms in Lectures 9-10.

Real-life applications of string problems: Search Engines & Information Retrieval, Spell Checkers & Auto-Correct Systems, Cybersecurity (detect malware signatures in files), Bioinformatics (align DNA sequences to identify mutations), Text Mining (extract meaningful patterns from large text datasets), Data Compression, Natural Language Processing (chatbots, language translation), Plagiarism Detection.

3.3 Classification based on the solvability of the problem

3.3.1 Solvable (Decidable) Problems

A solvable (decidable) problem is one for which an algorithm exists that:

- Always **terminates** after a finite number of steps.
- Provides a **correct** answer for any valid input.

All problems given so far as example are decidable problems.

3.3.2 Unsolvable (Undecidable) Problems

An unsolvable (undecidable) problem is one for which **no algorithm exists** that can always provide a correct solution for all valid inputs.

Example: Halting Problem: Given a computer program P and input data x, will P eventually halt on input x, or will it run forever? The problem was first introduced by Alan Turing in 1936, and he proved that it is undecidable.

Theorem (Undecidability of the Halting Problem): There is no algorithm that can solve the halting problem for all possible program-input pairs.

Proof by Contradiction (Turing's Proof, 1936, see[4])

Assumption: Suppose there exists an algorithm $H(P, x)$ that correctly determines whether any program P halts on input x.

$$H(P,x) = \begin{cases} \text{Yes}, & \text{if } P \text{ halts on input } x \\ \text{No}, & \text{if } P \text{ runs forever on input } x, \end{cases}$$ where P is an arbitrary program, and x is an arbitrary input

Construct the contradiction: Define a new program D (from Diagonalization) as follows:

$$D(P) = \begin{cases} \text{loop forever}, & \text{if } H(P,P) = \text{Yes} \\ \text{halt immediately}, & \text{if } H(P,P) = \text{No} \end{cases}$$

Contradiction: Consider running D on itself, $D(D)$:

- If $H(D, D) = \text{Yes}$, then D should loop forever by definition of D (contradiction).
- If $H(D, D) = \text{No}$, then D halts immediately (contradiction).

Conclusion: In both cases, we arrive at a logical contradiction. Therefore, our initial assumption that the halting algorithm $H(P, x)$ exists must be false. Thus, the halting problem is undecidable.

3.3.3 Semi-Decidable (Recursively Enumerable) Problems

A semi-decidable problem:

- Has an algorithm that will provide a Yes answer if the solution exists.
- Doesn't terminate if the answer is No.

Example: Theorem Proving in Logic: can a particular logical statement (called a theorem) be derived (or proven) from a given set of axioms and inference rules? A valid proof can eventually be found if it exists, but if no proof exists, the search could run forever.

4 Internal representation of data

In computers, all types of data are ultimately represented in binary form (sequences of 0s and 1s). This binary data is then interpreted by hardware and software according to the type of data being processed.

4.1 Representing numbers

Integers are typically represented using base-2 notation, and stored as a fixed number of bits:

- 8 bits (1 byte) → char
- 16 bits (2 bytes) → short int
- 32 bits (4 bytes) → int
- 64 bits (8 bytes) → long int

We defined in the previous section the problem PRIMES, that asks given an input number whether the number is prime or not. The problem is intensely used in cryptography, where very large numbers are used. These numbers are typically hundreds to thousands of bits long (4096 bits for high security RSA), to prevent brute-force attacks. With so large numbers, it is clear that number operations that we typically consider to take one computation step will need in practice more steps. We will discuss more about this in the following lecture.

4.2 Representing characters/strings

We said that computers represent all data in binary form. To store a character, an integer value (encoding) is assigned to the character. Strings are represented as sequences of characters followed by a null terminator (\0 in C/C++). Several standard encodings exist.

4.2.1 ASCII (American Standard Code for Information Interchange)

The ASCII encoding represents English characters using 7 bits, where 'A' is 65, 'a' is 97, and the rest of letter encodings are done increasingly, in lexicographic order, as detailed in Table1. These encodings are relevant for some of the text algorithms you will study later in this course.

4.2.2 Unicode Transformation Format (UTF-8, UTF-16, UTF-32)

Represents characters from most writing systems in the world (allowing for special characters, emojis, Chinese characters).

Unicode allows for variable-length encoding, using 1 to 4 bytes per character. It is backward compatible with ASCII (first 128 Unicode characters match ASCII exactly), and is efficient for text-heavy files (saves space for languages that use Latin alphabets). Table2 gives some example encodings, for a Latin character, a special western character and an emoticon.

Table 1: ASCII Encoding Table (0–127)

Dec	**Char**	**Dec**	**Char**	**Dec**	**Char**	**Dec**	**Char**
0	NUL	32	Space	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	’	71	G	103	g
8	BS	40	(	72	H	104	h
9	TAB	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	ETB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	|
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

Table 2: UTF-8 encoding example

Character	**Unicode Code Point**	**UTF-8 (Hexadecimal)**	**UTF-8 (Binary)**
A	U+0041	`0x41`	01000001
€(Euro)	U+20AC	`0xE2 0x82 0xAC`	11100010 10000010 10101100
😊 (Smiley Face)	U+1F60A	`0xF0 0x9F 0x98 0x8A`	11110000 10011111 10011000 10001010

4.3 Representing colors/images

Colors are usually represented using numeric codes that map to specific colors. There are several models that can be used.

1. **RGB (Red, Green, Blue) Model**

   Colors are represented using three values corresponding to red, green, and blue intensity (each from 0 to 255 in 8-bit representation). Stored as a 24-bit integer (8 bits per color).

   Example: Yellow in RGB:

   - Decimal: (255, 255, 0) (full red and green, no blue)
   - Hexadecimal: #FFFF00 (used in HTML/CSS styling)
   - Binary: 11111111 11111111 00000000

2. **CMYK (Cyan, Magenta, Yellow, Black) Model**

   This model is used in color printing. Each color channel ranges from 0 to 100%.

   Example: Pure yellow: C=0%, M=0%, Y=100%, K=0%

3. **Alpha Channel (Transparency)**

   Uses the RGB model, with an extra channel that represents the transparency level of a color. RGBA (Red, Green, Blue, Alpha: opacity level from 0 to 255).

   Example: Semi-transparent yellow: #FFFF0080

Images are collections of colored pixels. Raster Images (Bitmap Images) are represented as a grid of pixels (width×height). Common formats are JPG, PNG, BMP, GIF. Vector Images represent images using geometric shapes (points, lines, curves) stored as mathematical equations instead of pixels, and therefore are scalable without losing quality. Common vector image formats are SVG, EPS.

5 Algorithms

An algorithm is a well-defined, step-by-step sequence of instructions designed to perform a specific task or solve a particular problem. A good algorithm is not only **correct** but also **efficient** in terms of time (speed) and space (memory usage).

5.1 From problem to algorithm to implementation

The tendency with most Computer Science students is to, when given a problem, start right away with an **implementation** in a preferred programming language, and "figure out the details" on the go. The problem with this approach is that you may later discover that, e.g., the data structures you are using are not well suited for the problem you are trying to solve, and have to restart from scratch. Or you may have an incorrect or an inefficient solution.

We propose a five-step approach to solving problems:

1. **Problem**

   Understand what the problem asks. Clearly define the problem as Input-Output. Are there any constraints (time limits, memory limits)? Is the problem conceptually similar to something you already know? What does a valid solution look like? Take some examples.

2. **Algorithm**

   Think through potential solutions conceptually. Break the problem into smaller subproblems. Consider edge cases (empty inputs, maximum size inputs, negative numbers, etc.). Jot down ideas in natural language.

3. **Pseudocode**

   Write a high-level description of your algorithm in pseudocode. What data structures would you use? Clarify your thought process. Spot logical errors early.

4. **Implementation**

   Translate your pseudocode into real code using your preferred programming language. Stick to the logical structure developed earlier to avoid introducing new bugs.

5. **Assessment**

   Test with various input cases. Do the outputs make sense? Are there any errors? Optimize for time and space where necessary.

A **mnemonic** that can help you remember these steps is:
Please Always Plan Intelligently Ahead. Each letter corresponds to a step:

P	$\rightarrow$	Problem (Understand the problem thoroughly)
A	$\rightarrow$	Algorithm (Design a logical solution strategy)
P	$\rightarrow$	Pseudocode (Write the steps in a clear, structured way)
I	$\rightarrow$	Implementation (Translate pseudocode into actual code)
A	$\rightarrow$	Assessment (Test, debug, and optimize the solution)

Another shorter mnemonic is **Plan Ahead, Plan It All**.

5.2 Example: The Search Problem

Very often we need to search for a certain object (value) among a (finite) set of objects (values). In practice, you can think of searching for your car keys, your ID the night before or in the morning of an exam, searching for something on Google/Bing/DuckDuckGo etc., or searching for a keyword in a document (e.g., during an open-book exam). In all these cases, the time factor is important, so you want to execute the search in an efficient way.

If we are trying to solve a search problem on a computer, we need to make sure that we are using only the input that is provided, we are producing the desired output, and we take into account the possible corner cases.

5.2.1 Problem.

Let's assume the problem is to search for a nonnegative integer value in a given array of sorted data without duplicates.

Element-Search
Input: $(a_1, a_2, \ldots, a_n)$, where $a_i \in \mathbb{N}, a_i < a_{i+1}, \forall\, 1 \leq i \leq n-1$, and $k \in \mathbb{N}$ (the searched value);
Output: $i \in \mathbb{N} \cup \{-1\}$, s.t. $a_i = k$, if such an index exists, or -1 if k is not in a.

5.2.2 Algorithm.

We mentioned no restrictions in terms of time or space, so any correct search idea is good enough. What we can do is linear search: iterate through the array, checking at each step whether the current element is k or not. If k is found, return the current position. If there are no elements left in a and k has not been found, return -1.

5.2.3 Pseudocode.

Algorithm 1 Linear Search

Explanation of variables: **a**, the nonnegative integers array; **k**, the target element.

```
n ← a.size()
for i= 0..n-1 do
    if a[i] == k then
        return i
return -1
```

5.2.4 Implementation.

Linear Search Implementation

```
#include<iostream>
using namespace std;

int main() {
    int a[10], i, k, index;

    std::cout << "Enter 10 Numbers: ";
    for (i = 0; i < 10; i++)
        std::cin >> a[i];

    std::cout << "\nEnter a Number to Search: ";
    std::cin >> k;

    for (i = 0; i < 10; i++) {
        if (a[i] == k) {
            index = i;
            break;
        }
    }

    std::cout << "\nFound at Index No." << index;
    std::cout << endl;

    return 0;
}
```

5.2.5 Assessment.

Now, we have a working program with a hard-coded input. We start improving our code, and think of usability, e.g. defining functions. It is best to already think ahead and do this in the

first implementation.

Linear Search Implementation

```
int search(vector<int>&a, int k) {
    int n = a.size();
    for (int i = 0; i < n; i++)
        if (a[i] == k)
            return i;
    return -1;
}
```

5.2.6 Binary search

Can we do better? Of course! Given that the input array is sorted, we can start the search in the middle. If we found the target element, we return the index. If the current element is less than the target, we search to the right, otherwise we search to the left.

Algorithm 2 Binary Search

Input: A sorted array a of length n, a target value k
Output: The index of k if found, otherwise -1

```
low ← 0
high ← n − 1
while low ≤ high do
    mid ← ⌊(low + high)/2⌋                ▷ Find the middle index
    if a[mid] == k then
        return mid                        ▷ Element found at index mid
    else if a[mid] < k then
        low ← mid + 1                     ▷ Search in the right half
    else
        high ← mid − 1                    ▷ Search in the left half
return −1                                 ▷ Element not found
```

"Perhaps the most important principle for the good algorithm designer is to refuse to be content."[1]

Always ask yourselves: **"Can we do better?"**

References

[1] Alfred V Aho and John E Hopcroft. *The design and analysis of computer algorithms*. Pearson Education India, 1974.

[2] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 2009.

[3] Jon Kleinberg and Eva Tardos. *Algorithm design*. Pearson Education India, 2006.

[4] Alan Mathison Turing et al. On computable numbers, with an application to the entscheidungsproblem. *J. of Math*, 58(345-363):5, 1936.